

Jogo

Nome do arquivo: `jogo.c`, `jogo.cpp`, `jogo.pas`, `jogo.java`, `jogo.js` ou `jogo.py`

Uma empresa está desenvolvendo um aplicativo para celular que tem como objetivo estimular o gosto por matemática em jovens.

O aplicativo é um jogo, chamado Maior ou Menor, que sorteia um número inteiro e o jogador tem que adivinhar qual o número sorteado. O jogo é composto de uma ou mais rodadas. A cada rodada, o jogador digita um número e o aplicativo responde com:

- **menor** se o número digitado é maior do que o sorteado;
- **maior** se o número digitado é menor do que o sorteado; e
- **correto** se o número digitado é igual ao número sorteado.

O jogo termina quando o jogador acerta o número sorteado.

Dados o número sorteado e as tentativas de um jogador, você deve escrever um programa que simule o comportamento do aplicativo.

Entrada

A primeira linha contém um inteiro X , o número sorteado pelo aplicativo. Cada uma das linhas seguintes contém um inteiro T , indicando uma tentativa do jogador de acertar o número sorteado. Na última linha da entrada, $T = X$.

Saída

Para cada tentativa do jogador seu programa deve produzir uma linha na saída, contendo apenas uma palavra, que deve ser **menor** se o valor da tentativa é maior do que o número sorteado, **maior** se o valor da tentativa é menor do que o número sorteado ou **correto** se o valor da tentativa é igual ao número sorteado.

Restrições

- $-10\,000 \leq X \leq 10\,000$
- $-10\,000 \leq T \leq 10\,000$
- Haverá no máximo 1000 rodadas em cada jogo.

Informações sobre a pontuação

- Para um conjunto de casos de testes valendo 43 pontos, é garantido que o jogo termina em exatamente duas rodadas.
- Para um outro conjunto de casos de testes valendo 57 pontos, nenhuma restrição adicional.

Exemplos

Exemplo de entrada 1	Exemplo de saída 1
103	menor
1000	maior
1	menor
500	menor
200	maior
100	menor
110	menor
105	correto
103	

Exemplo de entrada 2	Exemplo de saída 2
10000	
10000	correto

Exemplo de entrada 3	Exemplo de saída 3
-2	menor
0	menor
-1	correto
-2	

Teste de redação

Nome do arquivo: `teste.c`, `teste.cpp`, `teste.pas`, `teste.java`, `teste.js` ou `teste.py`

Você é o mais novo estagiário numa empresa de inteligência artificial que está desenvolvendo um corretor automatizado de redações. Para testar o corretor, seu chefe solicitou que você escreva um programa que gere redações aleatórias – que por não fazerem sentido, devem receber a nota zero do corretor automatizado!

As redações geradas pelo seu programa devem obedecer aos seguintes requisitos:

- Cada palavra deve ser composta apenas por letras minúsculas.
- Cada palavra deve ter no mínimo uma letra e no máximo 10 letras.
- Duas palavras devem ser separadas por um espaço em branco.
- A redação deve ser composta por no mínimo N palavras e no máximo M palavras.
- A redação deve ser composta por no mínimo $M/2$ palavras distintas.

Entrada

A primeira linha contém dois inteiros N e M , indicando respectivamente o número mínimo e o número máximo de palavras da redação teste.

Saída

Seu programa deve produzir uma única linha, contendo a redação produzida.

Restrições

- $1 \leq N \leq M \leq 10\,000$
- M é par
- Apenas letras minúsculas não acentuadas são permitidas: `abcdefghijklmnopqrstuvwxyz`

Informações sobre a pontuação

- Para um conjunto de casos de testes valendo 33 pontos, $26 \leq M \leq 52$.
- Para outro conjunto de casos de testes valendo 67 pontos, nenhuma restrição adicional.

Exemplos

Exemplo de entrada 1	Exemplo de saída 1
2 10	melhor que machado de assis

Explicação do exemplo 1: A redação deve ter no mínimo duas e no máximo dez palavras, então uma redação com cinco palavras obedece ao critério do número de palavras. Além disso, a redação contém cinco palavras distintas, o que obedece ao critério de número de palavras distintas (mínimo de cinco). Então a redação produzida está correta.

Exemplo de entrada 2	Exemplo de saída 2
6 6	b c d e f g

Explicação do exemplo 2: A redação deve ter exatamente seis palavras, então a redação produzida tem o número correto de palavras. Além disso, a redação contém seis palavras distintas, mais do que o mínimo exigido (três). Então a redação produzida está correta.

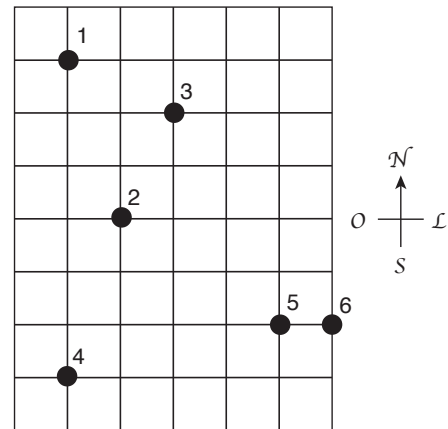
Exemplo de entrada 3	Exemplo de saída 3
6 8	aa bb aa bb aa bb

Explicação do exemplo 3: A redação deve ter no mínimo seis e no máximo oito palavras, então uma redação com seis palavras é correta. No entanto a redação produzida não tem o número mínimo de palavras distintas necessário (tem três, deveria ter no mínimo quatro), então a redação mostrada está INCORRETA.

Carro elétrico

Nome do arquivo: `carro.c`, `carro.cpp`, `carro.pas`, `carro.java`, `carro.js` ou `carro.py`

O mapa ao lado mostra o reino de Quadradônia. As estradas são representadas por linhas e as cidades por círculos numerados de 1 a 10. As estradas são igualmente espaçadas com distância de 100 km entre cada par de estradas, sendo orientadas em apenas duas direções: Norte-Sul e Leste-Oeste. Uma empresa de aluguel de carros em Quadradônia utiliza apenas carros elétricos. A *autonomia* de um carro elétrico é a distância que ele pode percorrer com uma carga de energia; após essa distância o carro deve ser carregado novamente para que possa ser utilizado. Há carregadores de energia em cada cidade e não há carregadores de energia fora das cidades. Entre cidades, os carros trafegam apenas pelas estradas e todos os carros têm a mesma autonomia.



Um vendedor deseja partir da cidade 1 e visitar todas as outras cidades, em qualquer ordem, mesmo que ele visite a mesma cidade mais de uma vez. Ele quer utilizar preferencialmente carros elétricos na sua viagem, mas se necessário viajará de avião se a distância para a próxima cidade for maior do que a autonomia do carro. Por exemplo, no mapa acima, se a autonomia for 300 km, o vendedor pode alugar um carro em 1 e visitar 3 e depois 2, mas não pode alcançar as outras cidades. Então ele pode viajar de avião até 5, alugar um carro visitar 6, depois viajar de avião até 4. Assim, são necessárias duas viagens de avião para ele visitar todas as cidades.

Dados o mapa da Quadradônia e a autonomia dos carros, determine qual o menor número de viagens de avião que são necessárias para que o viajante visite todas as cidades, partindo da cidade 1.

Entrada

A primeira linha contém dois inteiros X e Y , indicando respectivamente o número de estradas na direção Oeste-Leste e estradas na direção Norte-Sul. As estradas são numeradas de 1 a X na direção Oeste-Leste e de 1 a Y na direção Norte-Sul. A segunda linha contém dois inteiros N e A , indicando respectivamente o número de cidades e a autonomia dos carros, em quilômetros. As cidades são numeradas de 1 a N . Cada uma das N linhas seguintes contém um par de inteiros x_i e y_i , indicando a posição da cidade de número i , para $1 \leq i \leq N$.

Saída

Seu programa deve produzir uma única linha, contendo um único inteiro, o menor número de viagens de avião necessárias para que o vendedor visite todas as cidades.

Restrições

- $1 \leq X \leq 100\,000$
- $1 \leq Y \leq 100\,000$
- $2 \leq N \leq 1\,000$
- $1 \leq A \leq 150\,000$
- $1 \leq x_i \leq X$
- $1 \leq y_i \leq Y$

Informações sobre a pontuação

- Para um conjunto de casos de testes valendo 37 pontos, $N = 2$.
- Para outro conjunto de casos de testes valendo 32 pontos, $Y = 1$, e é garantido que as cidades são dadas em ordem crescente de X (isto é, $x_1 < x_2 < \dots < x_n$).
- Para um outro conjunto de casos de testes valendo 31 pontos, nenhuma restrição adicional.

Exemplos

Exemplo de entrada 1	Exemplo de saída 1
7 9 6 300 2 2 3 5 4 3 2 8 6 7 7 7	2

Explicação do exemplo 1: Este é o exemplo do enunciado.

Exemplo de entrada 2	Exemplo de saída 2
7 9 6 200 2 2 3 5 4 3 2 8 6 7 7 7	4

Explicação do exemplo 2: Como a autonomia é 200 km, a única viagem de carro possível é entre as cidades 5 e 6. O vendedor pode por exemplo viajar de avião de 1 para 3, depois de 3 para 2, depois de 2 para 4, depois de 4 para 5, alugar um carro e visitar 6, para um total de quatro viagens de avião.

Dígitos

Nome do arquivo: `digitos.c`, `digitos.cpp`, `digitos.pas`, `digitos.java`, `digitos.js` ou `digitos.py`

Joãozinho te propôs o seguinte desafio: ele escolheu dois inteiros A e B , com $1 \leq A \leq B \leq 10^{1000}$, e escreveu na lousa todos os inteiros entre A e B , em sequência, porém colocando um espaço após cada dígito, de forma a não ser possível ver quando um número termina ou começa. Por exemplo, se Joãozinho escolher $A = 98$ e $B = 102$, ele escreveria a sequência "9 8 9 9 1 0 0 1 0 1 1 0 2".

Seu desafio é: dada a lista de dígitos escritos na lousa, encontrar os valores de A e B . Caso exista mais de uma possibilidade para os valores que geraria a lista, você deve encontrar uma em que o valor de A é o menor possível.

É garantido que a lista de dígitos da lousa tem no máximo tamanho 1000.

Entrada

A primeira linha da entrada contém um único inteiro N , indicando o número de dígitos. A segunda linha contém N inteiros d_i , indicando os dígitos escritos.

Saída

Imprima o menor valor possível de A .

Restrições

- $1 \leq N \leq 1000$
- $0 \leq d_i \leq 9$

Informações sobre a pontuação

- Para um conjunto de casos de testes valendo 21 pontos, $1000 \leq A \leq B \leq 9999$.
- Para outro conjunto de casos de testes valendo 23 pontos, $B = A + 1$.
- Para outro conjunto de casos de testes valendo 40 pontos, $A, B < 10^6$.
- Para outro conjunto de casos de testes valendo 16 pontos, nenhuma restrição adicional.

Exemplos

Exemplo de entrada 1 6 1 2 3 1 2 4	Exemplo de saída 1 123
Exemplo de entrada 2 6 8 9 1 0 1 1	Exemplo de saída 2 8

Restaurante de pizza

Nome do arquivo: `pizza.c`, `pizza.cpp`, `pizza.pas`, `pizza.java`, `pizza.js` ou `pizza.py`

Um amigo seu acabou de se mudar para Linearlandia. Apesar de recém chegado, ele decidiu montar um negócio, um restaurante de Pizza.

Seu amigo está muito feliz na nova empreitada, contudo, com muito medo de errar. Além do preparo da pizza, o restaurante deve se preocupar com a caixa para entrega (que são retangulares), que deverá ser a mesma para todas as pizzas, e com o corte das fatias, que será automatizado e igual para todas as pizzas produzidas.

No momento seu amigo está planejando qual será o tamanho das pizzas (que serão todas círculos perfeitos de um único tamanho), e também qual será o ângulo interno de cada fatia (em graus). Além disso, ele encontrou uma loja de caixas de pizza com ótimos preços, mas não sabe se as caixas são adequadas para as pizzas que ele vai produzir.

A restrição para a caixa de pizza é que a pizza caiba dentro da caixa (mesmo que com alguma folga); a restrição para o ângulo de corte é que todas as fatias sejam de mesmo tamanho (mesmo que seja uma só fatia).

Dados as dimensões da caixa de pizza, o raio da pizza e o ângulo interno da fatia em graus, você deve escrever um programa para determinar se a caixa e o ângulo escolhidos satisfazem às restrições.

Entrada

A entrada é composta por quatro linhas, contendo respectivamente os números inteiros A , B , R e G . Os inteiros A e B são as dimensões da caixa de pizza, o inteiro R é o raio da pizza e o inteiro G é o ângulo interno das fatias de pizza.

Saída

Seu programa deve produzir uma única linha na saída, contendo um único caractere, que deve ser **S** se os dados satisfazem às restrições ou **N** caso contrário.

Restrições

- $1 \leq A, B, R \leq 10^9$;
- $1 \leq G \leq 360$.

Informações sobre a pontuação

- Para um conjunto de testes valendo 13 pontos, $A = B$;
- Para um conjunto de testes valendo 26 pontos, $G = 60$;
- Para um conjunto de casos de testes valendo outros 61 pontos, nenhuma restrição adicional.

Exemplos

Exemplo de entrada 1	Exemplo de saída 1
4 10 2 60	S

Explicação do exemplo 1: A pizza cabe na caixa e a escolha de ângulo divide a mesma igualmente.

Exemplo de entrada 2	Exemplo de saída 2
4 4 3 60	N

Explicação do exemplo 2: O raio da pizza é 3, logo, não cabe em uma caixa de lados de tamanho 4.

Exemplo de entrada 3	Exemplo de saída 3
10 10 5 25	N

Explicação do exemplo 3: O ângulo da fatia é 25, logo, não resulta em fatias iguais.

Pilhas de moedas

Nome do arquivo: `pilhas.c`, `pilhas.cpp`, `pilhas.pas`, `pilhas.java`, `pilhas.js` ou `pilhas.py`

Flávia possui várias moedas em sua coleção, que estão organizadas em N pilhas, cada pilha com um certo número de moedas. Vamos chamar o número de moedas de uma pilha de *altura* da pilha.

A garota pretende adicionar algumas moedas à sua coleção, de forma que cada moeda nova deve ser adicionada em uma das pilhas existentes. As moedas originais, porém, devem permanecer nas suas pilhas.

Flávia está se perguntando agora: qual o número mínimo de moedas que ela deve adicionar à coleção para que, considerando os valores de todas as N novas alturas de pilhas, a quantidade de números distintos seja no máximo K ?

Por exemplo, se a lista de alturas inicialmente é $(3, 5, 8, 4, 5, 8)$, temos que existem 4 valores distintos de alturas: 3, 4, 5 e 8. Se $K = 2$, poderíamos, com 3 moedas novas, adicionar duas na pilha de índice 1, e uma na pilha de índice 4. Assim, a lista de alturas ficará $(5, 5, 8, 5, 5, 8)$, que possui apenas dois valores distintos de alturas: 5 e 8.

Note que, se inicialmente a lista de alturas já tem no máximo K valores distintos, Flávia já estaria feliz, e não iria precisar de nenhuma moeda nova.

Entrada

A primeira linha da entrada contém um dois inteiros separados por espaços N , indicando o número de pilhas e K , indicando o número máximo de valores distintos. A segunda linha contém N inteiros P_i , indicando as alturas das pilhas.

Saída

Imprima a menor quantidade adicional de moedas.

Restrições

- $1 \leq N \leq 500$
- $1 \leq K \leq N$
- $1 \leq v_i \leq 500$

Informações sobre a pontuação

- Para um conjunto de casos de testes valendo 13 pontos, $K = 1$.
- Para outro conjunto de casos de testes valendo 21 pontos, $K = 2$.
- Para outro conjunto de casos de testes valendo 28 pontos, $K, N, v_i \leq 50$.
- Para outro conjunto de casos de testes valendo 38 pontos, nenhuma restrição adicional.

Exemplos

Exemplo de entrada 1	Exemplo de saída 1
6 2 5 3 8 4 5 8	3

Exemplo de entrada 2	Exemplo de saída 2
6 3 5 3 8 4 5 8	1

Caravana

Nome do arquivo: `caravana.c`, `caravana.cpp`, `caravana.pas`, `caravana.java`, `caravana.js` ou `caravana.py`

No deserto da Nlogônia, uma longa caravana de camelos carregados de especiarias está parada num oásis para descansar. O chefe da caravana notou que alguns camelos pareciam mais cansados do que os outros, e descobriu que cada camelo estava carregando um peso diferente, de forma que alguns camelos carregam um peso muito maior do que outros e portanto se cansam mais.

Aproveitando a parada para descanso, o chefe da caravana quer redistribuir as especiarias entre os camelos, de forma que todos os camelos carreguem exatamente o mesmo peso.

Dados os pesos carregados por cada camelo antes da parada, escreva um programa que determine, para cada camelo, qual o peso que deve ser retirado ou adicionado, para que todos carreguem exatamente o mesmo peso.

Entrada

A primeira linha contém um inteiro N , o número de camelos na caravana. Os camelos são numerados de 1 a N . Cada uma das linhas seguintes contém um inteiro P_i , o peso que o camelo de número i carregava antes da parada. Os camelos são dados em ordem crescente de numeração.

Saída

Para cada camelo da caravana, seu programa deve produzir uma linha, o valor que deve ser adicionado ou retirado desse camelo para que todos os camelos carreguem o mesmo peso. A ordem dos camelos na saída deve ser a mesma ordem dada na entrada. Para todos os casos de teste o peso que cada camelo deve carregar é um número inteiro.

Restrições

- $1 \leq N \leq 1\,000$
- $1 \leq P_i \leq 10\,000$ para $1 \leq i \leq N$

Exemplos

Exemplo de entrada 1 3 100 104 108	Exemplo de saída 1 4 0 -4
Exemplo de entrada 2 5 30 40 23 5 32	Exemplo de saída 2 -4 -14 3 21 -6

Exemplo de entrada 3	Exemplo de saída 3
3	0
10000	0
10000	0
10000	

Dona Minhoca

Nome do arquivo: `minhoca.c`, `minhoca.cpp`, `minhoca.pas`, `minhoca.java`, `minhoca.js` ou `minhoca.py`

Dona Minhoca construiu uma bela casa, composta de N salas conectadas por $N - 1$ túneis. Cada túnel conecta exatamente duas salas distintas, e pode ser percorrido em qualquer direção. A casa de dona Minhoca foi construída de modo que, percorrendo os túneis, é possível partir de qualquer sala e chegar a qualquer outra sala da casa.

Para deixar sua casa mais segura, Dona Minhoca decidiu instalar radares anti-furto em algumas das salas. Ela comprou K radares, e deve agora decidir em quais salas colocará um radar. Além disso, todos radares terão um *raio de alcance*, cujo valor R também deve ser decidido. Quando um radar com raio de alcance R é instalado na sala s , todas as salas com distância menor ou igual a R da sala s (incluindo a própria s) ficam sob o alcance do radar, e estarão protegidas.

Devido à política estranha de cobrança da empresa de radares, todos os K radares devem ter o mesmo raio de alcance. Dona Minhoca então se pergunta: qual seria o menor valor possível para R , tal que, se o raio de alcance dos radares for R , é possível escolher K salas para instalar os radares de forma que todas as N salas estejam protegidas?

Entrada

A primeira linha da entrada contém dois inteiros N e K , indicando o número de salas, e de radares que Dona Minhoca possui. As $N - 1$ linhas seguintes contém dois inteiros a_i e b_i cada, indicando que existe um túnel conectando essas duas salas.

Saída

Seu programa deve produzir uma única linha, contendo um único inteiro, o menor valor possível para R .

Restrições

- $1 \leq N \leq 300000$
- $1 \leq K < N$
- $a_i \neq b_i$

Informações sobre a pontuação

- Para um conjunto de casos de testes valendo 25 pontos, $K = 1$
- Para outro conjunto de casos de testes valendo 17 pontos, o túnel i conecta as salas i e $i + 1$ ($1 \leq i \leq N - 1$). Ou seja, a casa possui o formato de uma linha reta.
- Para outro conjunto de casos de testes valendo 17 pontos, $N, K \leq 100$
- Para outro conjunto de casos de testes valendo 41 pontos, nenhuma restrição adicional.

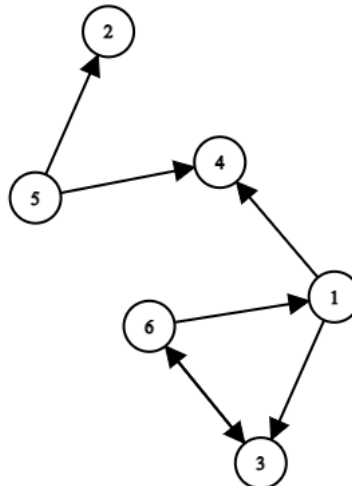
Exemplos

Exemplo de entrada 1 6 1 1 2 2 3 3 4 4 5 4 6	Exemplo de saída 1 2
Exemplo de entrada 2 6 2 1 2 2 3 3 4 4 5 4 6	Exemplo de saída 2 1

Rodovia

Nome do arquivo: `rodovia.c`, `rodovia.cpp`, `rodovia.pas`, `rodovia.java`, `rodovia.js` ou `rodovia.py`

O reino de Nlogonia é composto por N cidades, numeradas de 1 a N , e M rodovias **direcionadas**, ou seja, é possível usar a rodovia (x, y) para ir da cidade x à cidade y , porém não na outra direção.



Vamos definir o valor da *conectividade* do reino como o número de pares ordenados (x, y) , com $x \neq y$, tais que é possível viajar de x a y (talvez indiretamente, passando por outras cidades intermediárias pelo caminho). Na figura acima, por exemplo, o valor da conectividade é 11, sendo que os pares em questão são: $(1, 3)$, $(1, 4)$, $(1, 6)$, $(3, 1)$, $(3, 4)$, $(3, 6)$, $(5, 2)$, $(5, 4)$, $(6, 1)$, $(6, 3)$ e $(6, 4)$.

O governo de Nlogonia está planejando construir uma única nova rodovia (A, B) , também direcionada. Muitas discussões estão sendo feitas para escolher a rodovia ideal, porém no momento, o maior receio é se há alguma possibilidade de ser feita uma escolha que seja considerada redundante pelos habitantes do reino. Em particular, foi dada a você a tarefa de descobrir se existe algum par (A, B) de cidades tal que:

- $A \neq B$
- Não existe nenhuma rodovia (x, y) originalmente no reino, com $x = A$ e $y = B$.
- Caso adicionarmos a rodovia (A, B) , o valor da conectividade do reino permanecerá o mesmo.

Também foi pedido que, caso existam pares que cumpram todas as condições, você deve informar algum deles. Caso tenha mais de um par válido, você pode escolher qualquer um deles.

Entrada

A primeira linha da entrada contém dois inteiros N e M , indicando o número de cidades e rodovias. Seguem M linhas contendo dois inteiros x_i e y_i cada, indicando que existe uma rodovia que pode ser usada para viajar da cidade x_i à cidade y_i .

Saída

Caso exista algum par que satisfaça todas as condições, seu programa deve imprimir qualquer um desses pares, em uma única linha. Caso contrário, imprima -1 .

Restrições

- $1 \leq N \leq 200000$
- $1 \leq M \leq 400000$
- $x_i \neq y_i$
- Nenhuma rodovia é dada mais de uma vez na entrada, ou seja, $(x_i, y_i) \neq (x_j, y_j)$, se $i \neq j$. Note porém que é possível que ambas as rodovias (x, y) e (y, x) sejam dadas.

Informações sobre a pontuação

- Para um conjunto de casos de testes valendo 31 pontos, vale que a conectividade inicial do reino é igual a $N * (N - 1)$. Ou seja, existe algum caminho entre todos os pares de cidades.
- Para outro conjunto de casos de testes valendo 33 pontos, vale que se existe uma rodovia de x para y , então também existe uma rodovia de y para x .
- Para outro conjunto de casos de testes valendo 36 pontos, nenhuma restrição adicional.

Exemplos

Exemplo de entrada 1 4 3 1 2 2 4 1 4	Exemplo de saída 1 -1
Exemplo de entrada 2 4 4 1 2 2 4 1 4 4 3	Exemplo de saída 2 2 3

Quadrado

Nome do arquivo: `quadrado.c`, `quadrado.cpp`, `quadrado.pas`, `quadrado.java`, `quadrado.js` ou `quadrado.py`

Um *quadrado fantástico* é um conjunto de números inteiros positivos dispostos em N linhas por N colunas tal que:

- Não há números repetidos no quadrado.
- A média dos números em cada linha é um número inteiro que está presente na linha.
- A média dos números em cada coluna é um número inteiro que está presente na coluna.

Entrada

A primeira e única linha da entrada contém um número inteiro N , indicando a dimensão do quadrado.

Saída

Seu programa deve produzir N linhas, cada uma contendo N números inteiros X_i , representando um quadrado fantástico.

Restrições

- $1 \leq N \leq 40$
- $1 \leq X_i \leq 1000000$

Informações sobre a pontuação

- Para um conjunto de casos de testes valendo 44 pontos, $1 \leq N$ é ímpar.
- Para outro conjunto de casos de testes valendo 56 pontos, nenhuma restrição adicional.

Exemplos

Exemplo de entrada 1 1	Exemplo de saída 1 1
Exemplo de entrada 2 2	Exemplo de saída 2 -1
Exemplo de entrada 3 3	Exemplo de saída 3 1 2 3 4 5 6 7 8 9

Exemplo de entrada 4	Exemplo de saída 4
4	1 2 3 6 7 8 9 12 13 14 15 18 31 32 33 36